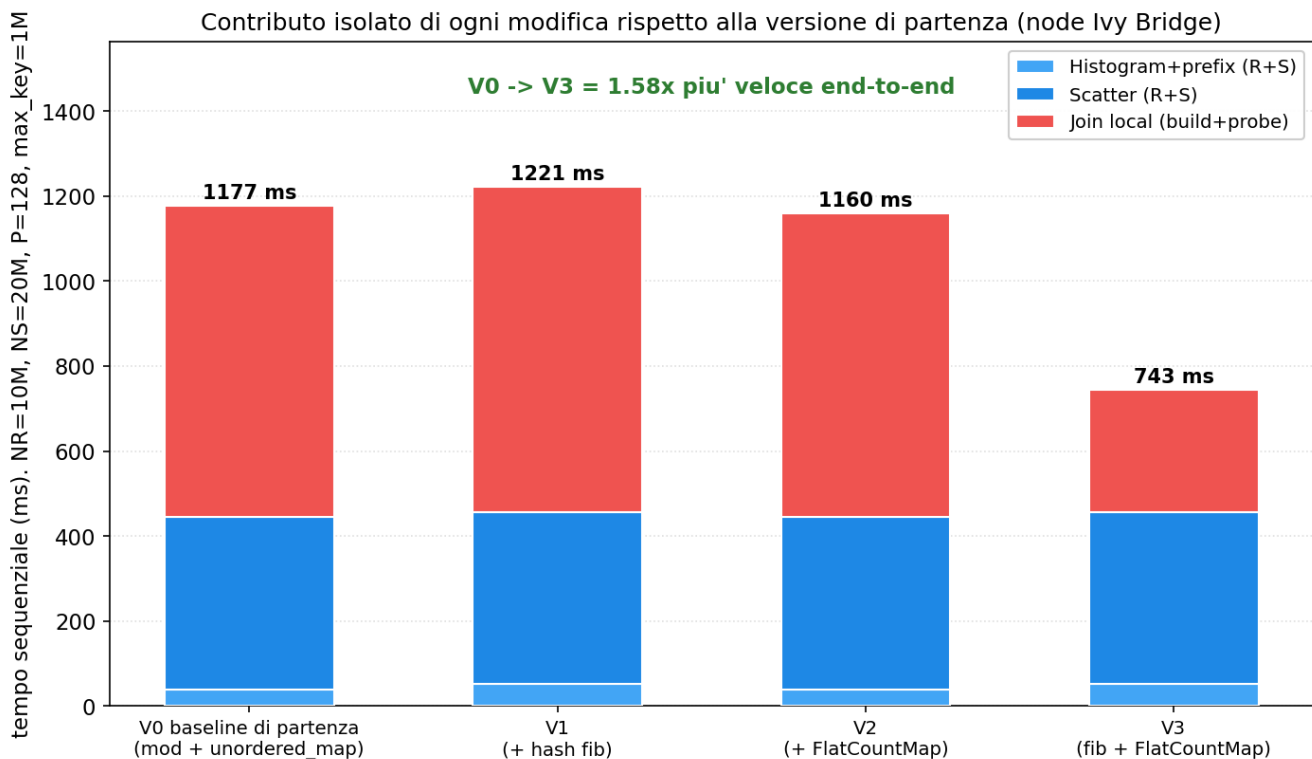


## Modulo 2: esperimenti aggiuntivi

Esperimenti a supporto del report del Modulo 2 (partitioned hash join con duplicati). Per ciascuno il grafico e i punti principali. Misure sul nodo Ivy Bridge (Xeon E5-2640 v2), lo stesso tipo di nodo del report.

| Esperimento                               | Riferimento nel report                 |
|-------------------------------------------|----------------------------------------|
| Esp. 1: baseline vs versione consegnata   | Join, hash di Fibonacci e FlatCountMap |
| Esp. 2: FlatCountMap                      | Join, struttura dati e padding a 64 B  |
| Esp. 3: barriera vs thread pool           | Thread team con barriera               |
| Esp. 4: distribuzione del carico nel join | Join, cyclic distribution              |
| Esp. 5: histogram memory-bound            | Histogram, $I = 0.125$                 |
| Esp. 6: legge di Amdahl                   | Analisi di scalabilità                 |

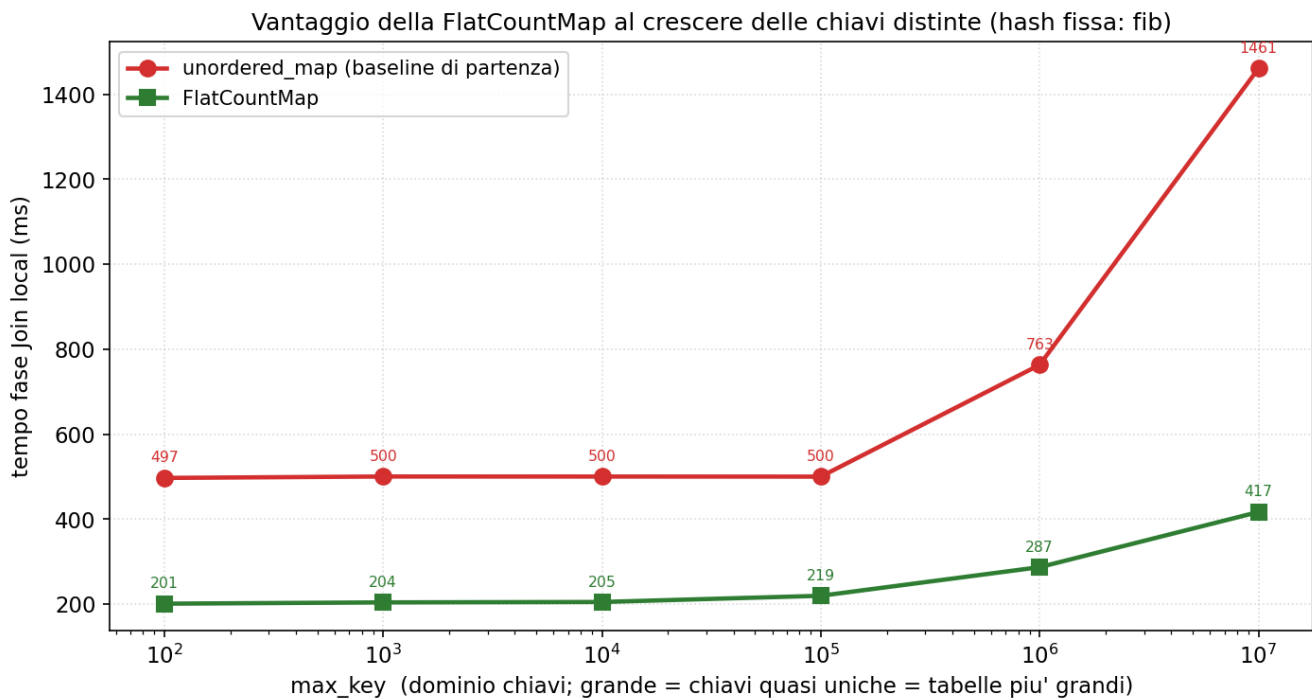
## Esp. 1: baseline vs versione consegnata



Le quattro combinazioni {mod, fib} per {unordered\_map, FlatCountMap}.

- Fase join: V0 732 ms, V1 (+fib) 765, V2 (+FlatCountMap) 714, V3 (fib + FlatCountMap) 286 ms.
- La FlatCountMap da sola migliora il join del 2% e la hash da sola non lo modifica; il crollo avviene solo con entrambe. Costituiscono una coppia obbligata, non due modifiche additive.
- La ragione è nei bit impiegati. La FlatCountMap ricava lo slot iniziale dai bit bassi della chiave (key & mask, hash identità), mentre fib ricava la partizione dai bit alti del prodotto  $(k_{lo} \wedge k_{hi}) * A32$ , che dipendono da tutta la chiave: le due funzioni leggono bit scorrelati, quindi dentro una partizione i bit bassi restano vari e le chiavi coprono tutti gli slot.
- Con mod la partizione è  $key \& (P-1)$ , cioè quegli stessi bit bassi: nella partizione p sono fissi a p, quindi gli slot iniziali raggiungibili sono solo p, p+P, p+2P, ..., uno su P. Nel bench (P=128, tabella da 262144 slot dimensionata sulle tuple R della partizione) le circa 7800 chiavi distinte si ammassano su 2048 slot iniziali invece che su tutti: il linear probing allunga le catene e la FlatCountMap perde il vantaggio, con il join a 714 ms contro i 286 di fib.

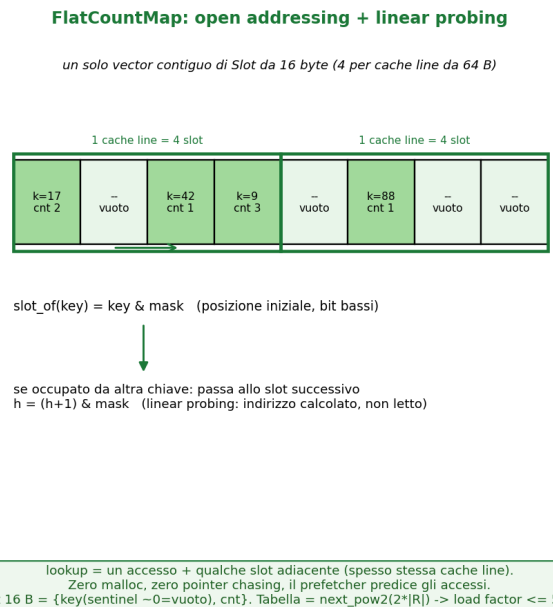
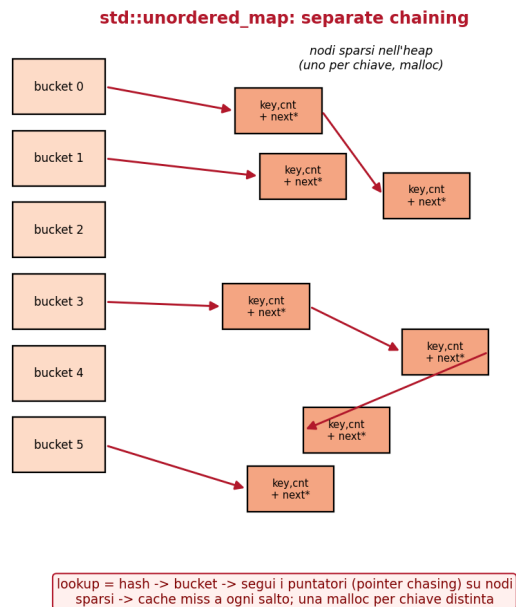
## Esp. 1: baseline vs versione consegnata



A hash fissa, il vantaggio della FlatCountMap cresce con le chiavi distinte.

- A hash fissata (fib), il vantaggio della FlatCountMap cresce con il numero di chiavi distinte: 2.5x a max\_key=100 (201 vs 497 ms), 3.5x a 10M (417 vs 1461 ms).
- L'unordered\_map alloca un nodo sull'heap per ogni chiave distinta: più chiavi uniche comportano più nodi sparsi e più pointer chasing. La FlatCountMap è un array contiguo, prevedibile per il prefetcher.
- Histogram e scatter restano invariati fra le varianti (circa 52 e 405 ms): le modifiche interessano solo hash e join.

## Esp. 2: FlatCountMap

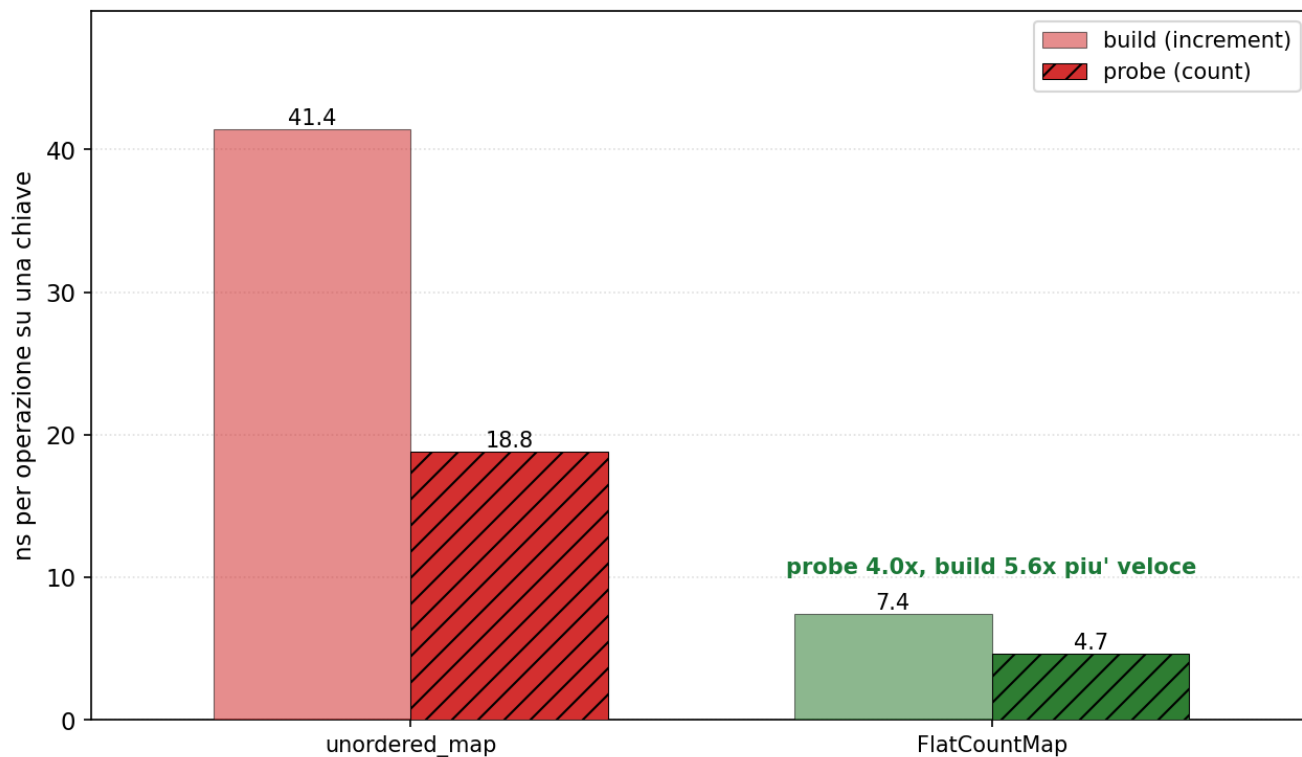


Le due strutture dati: separate chaining vs open addressing.

- L'unordered\_map adotta separate chaining: un array di bucket con liste di nodi allocati singolarmente sull'heap.
- La FlatCountMap adotta open addressing con linear probing: un unico vector contiguo, con Slot da 16 B, 4 per cache line.
- Ne conseguono l'assenza di allocazioni per chiave, buona località di cache e nessun pointer chasing, poiché lo slot successivo è calcolato anziché letto da un puntatore.

## Esp. 2: FlatCountMap

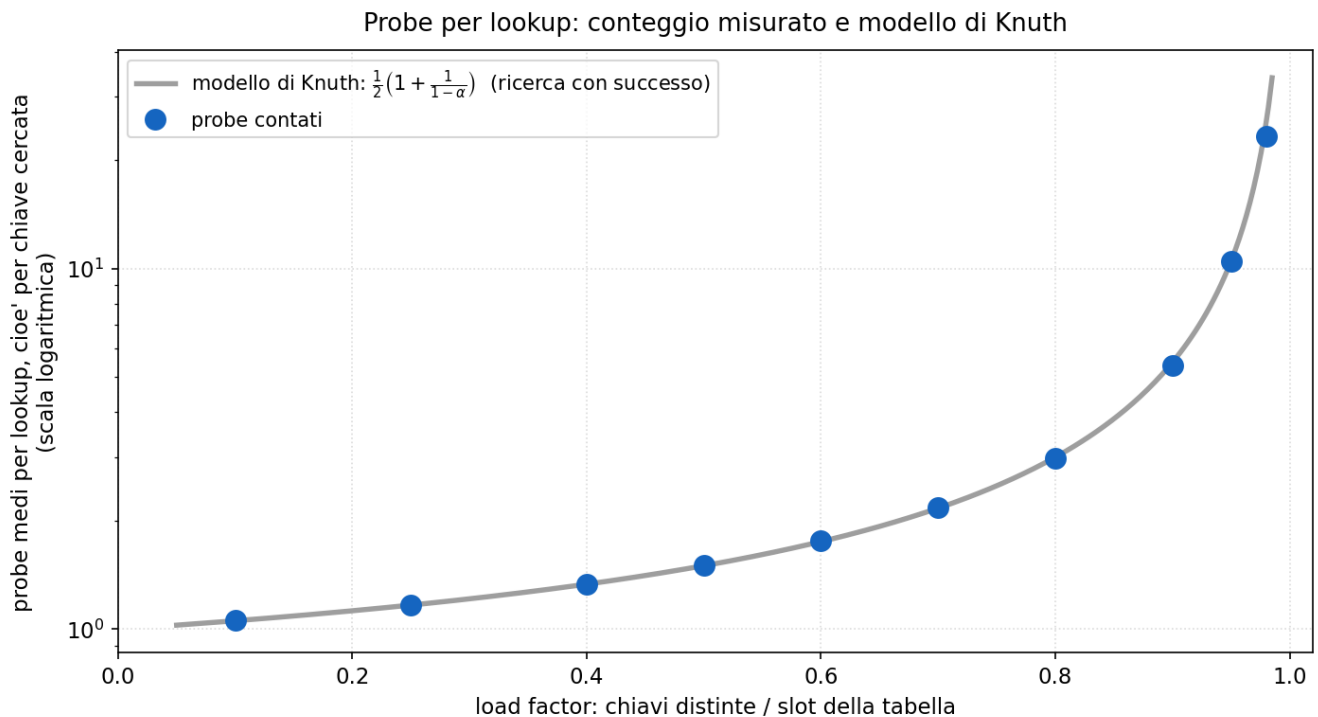
FlatCountMap vs unordered\_map: tempo per operazione (distinct=20k, single core, Ivy Bridge)



Tempo medio per operazione su una chiave: FlatCountMap vs unordered\_map.

- L'unità è il tempo di una singola operazione su una chiave (ns): il probe è una lookup, il build un increment.
- Il probe costa 18.8 ns con l'unordered\_map e 4.7 ns con la FlatCountMap (fattore 4); il build 41 vs 7 ns.
- Il vantaggio deriva dalla struttura contigua, con accessi locali, rispetto al pointer chasing dell'unordered\_map.

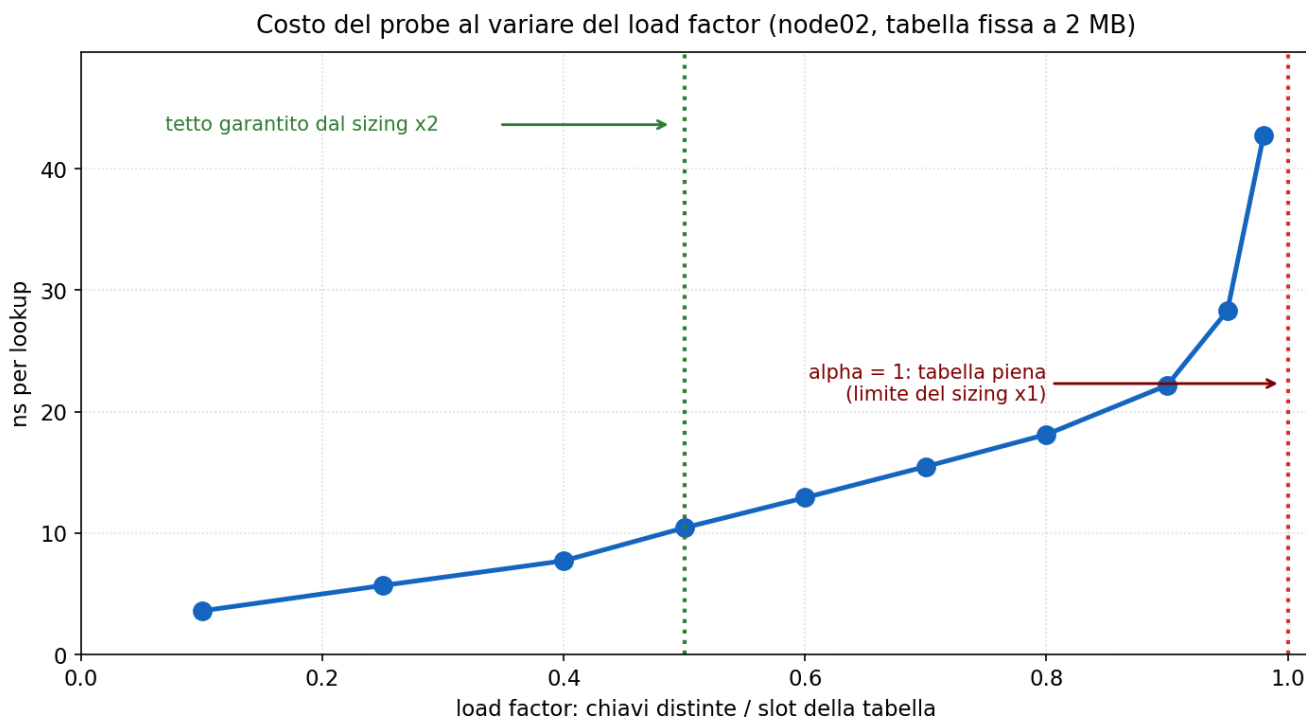
## Esp. 2: FlatCountMap — probe per lookup



Probe medi per lookup, cioè per singola chiave cercata: il bench cerca ogni chiave esattamente una volta. La curva grigia è il modello di Knuth per il linear probing con ricerca con successo, cioè un mezzo per  $(1 + 1/(1 - \alpha))$ ; i punti blu sono i probe effettivamente contati.

- Il load factor indica quanto è piena la tabella: chiavi distinte / slot.
- Knuth modella quanti slot deve visitare il linear probing per trovare una chiave, in funzione del solo riempimento.
- Questa analisi mostra la previsione teorica di riferimento per questa struttura, dividendo la quantità di passi che svolge l'algoritmo dal costo di un singolo passo.
- Dal grafico si nota come la quantità di probe coincida con il modello di Knuth entro il 2-9%, di conseguenza la discrepanza sui tempi dipende dal costo del singolo probe (prossimo grafico).

## Esp. 2: FlatCountMap — costo del probe e scelta del sizing



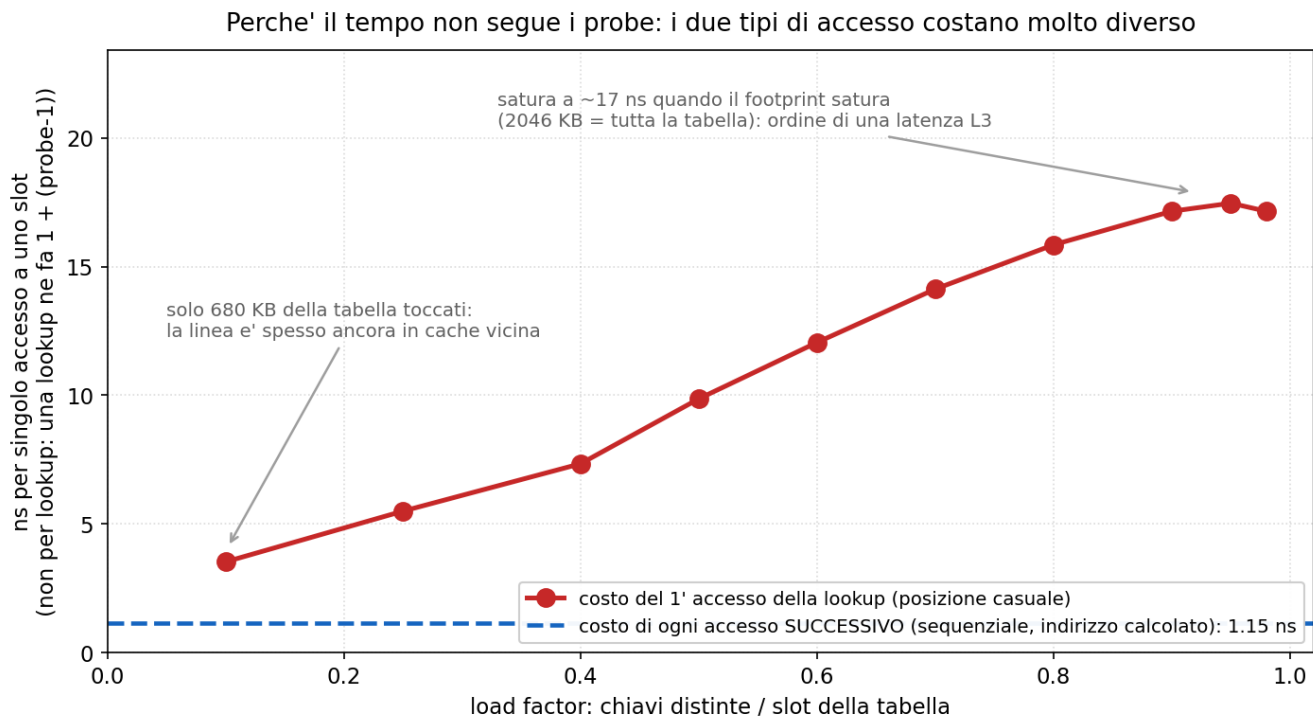
Costo di una lookup su node02 al variare del riempimento. La tabella è fissa a  $2^{17}$  slot da 16 B = 2 MB in tutte e dieci le misure e sta sempre dentro L3 (20 MB): varia solo il numero di chiavi distinte. Il degrado è quindi attribuibile al riempimento e non alla residenza in cache (grafico successivo).

- Il probe passa da 3.6 ns (riempimento 10%) a 10.4 (50%), 22.2 (90%), 42.7 (98%): con il linear probing gli slot occupati si aggregano in cluster e le catene si allungano.
- Il costo cresce già prima del 50%: 2.9x da 10% a 50% (2.7x in un re-run di controllo). Il tratto piatto termina verso il 25%.
- Il tetto di 0.5 non deriva dalla forma della curva, che a 0.5 non presenta alcun ginocchio, ma dalla regola di dimensionamento. `slot_of` ricava la posizione con `key & mask`, che sostituisce il modulo solo se il numero di slot è una potenza di due: la dimensione della tabella è quindi vincolata a essere una potenza di due.
- La tabella viene dimensionata al più piccolo valore potenza di due che sia almeno  $m$  volte il numero di record  $r$ , cioè  $n = \text{next\_pow2}(m \times r)$ , con  $m = 2$  nel codice. Nel caso peggiore le chiavi sono tutte distinte, quindi quelle inserite sono  $r$  e il riempimento vale  $r/n$ . Poiché  $n$  è per costruzione almeno  $m \times r$ , il riempimento è al più  $r/(m \times r) = 1/m$ .
- Il tetto  $1/m$  viene raggiunto quando  $n$  coincide con  $m \times r$ , cioè quando  $r$  è una potenza di due e `next_pow2` non arrotonda nulla. I tetti sono quindi 1.000 con x1, 0.500 con x2, 0.250 con x4, e passare da un sizing al successivo raddoppia la memoria.
- Con x1 il tetto è 1.000, cioè tabella piena. In quel caso `count()` di una chiave assente non termina: il `while` esce solo su slot vuoto o su chiave uguale, e con la tabella piena non si verifica nessuna delle due condizioni. Nel `join` la chiave assente è lo scenario ordinario, una chiave di `S` che non compare in `R`. Verificato eseguendolo.
- Fra i sizing che garantiscono la terminazione, x2 è quello con la minima occupazione di memoria: alloca 2 volte i record, contro le 4 volte di x4. Il riempimento più basso di x4 (0.250) non è un costo minore, è memoria in più spesa per stare più lontani dal tetto.
- x4 non viene adottato perché al punto operativo non produce guadagno. Il confronto diretto a parità di dati misura 4.681 ns con x2 e 4.648 ns con x4, cioè lo 0.7%, a fronte del doppio della

memoria. Nella configurazione consegnata (NR=10M, P=512, max\_key=1M) ogni partizione ha 19531 record e circa 1953 chiavi distinte, dunque 10 record per chiave: il riempimento operativo con x2 è 0.030 e la tabella resta vuota al 97%, ben dentro il tratto piatto della curva. Il vantaggio di x4 (probe da 10.4 a 5.7 ns) si materializza solo nel caso peggiore a zero duplicati, che questo carico non raggiunge.



## Esp. 2: FlatCountMap — perché il tempo non segue i probe

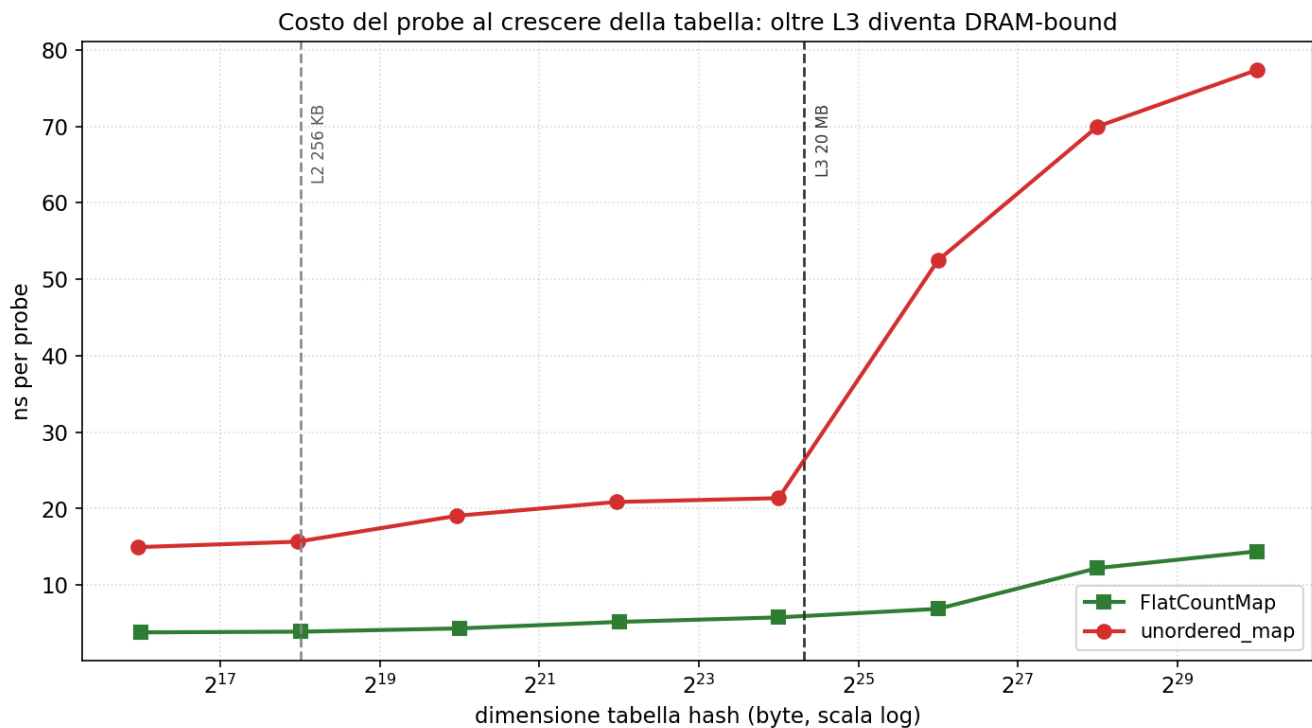


Decomposizione del costo di una lookup nelle sue due componenti. Una lookup è un accesso iniziale in posizione casuale (key & mask), più (probe - 1) accessi di seguito, sequenziali e con l'indirizzo calcolato. L'asse verticale riporta il costo di un singolo accesso, non di una lookup intera: il costo della lookup completa è il grafico precedente.

- Il modello è  $ns = C_{\text{primo}}(\text{riempimento}) + (\text{probe} - 1) \times C_{\text{seguito}}$ .
- Per riempimento oltre 0.90 il footprint è fermo (2035 -> 2046 KB), quindi  $C_{\text{primo}}$  è costante e si cancella nella differenza fra due punti di quella regione:  $ns(0.98) - ns(0.90) = [\text{probe}(0.98) - \text{probe}(0.90)] \times C_{\text{seguito}}$ , da cui  $(42.749 - 22.173) / (23.296 - 5.375) = 1.148$  ns. È una pendenza misurata dove l'altro termine è fermo e non dipende dal modello.
- Noto  $C_{\text{seguito}}$ ,  $C_{\text{primo}}$  si ottiene per differenza punto per punto:  $C_{\text{primo}} = ns - (\text{probe} - 1) \times C_{\text{seguito}}$ . Per riempimento 0.50:  $10.439 - 0.505 \times 1.148 = 9.86$  ns.
- Il costo del primo accesso cresce col footprint toccato (680 KB a 0.10, 2046 KB a 0.98) e satura a circa 17 ns dove satura il footprint, valore dell'ordine di una latenza L3.
- Ne segue la forma di entrambe le curve. A riempimento basso i probe sono circa 1 e la lookup costa quanto il primo accesso, che però sta crescendo: da 0.10 a 0.50 il tempo sale del 190% contro il 42% dei probe. A riempimento alto le catene si allungano ma gli accessi aggiuntivi costano 1.15 ns: da 0.90 a 0.98 i probe crescono del 333% e il tempo del 93%.
- Il basso costo di un accesso successivo non dipende dalla permanenza nella stessa cache line: la catena media è 5.4 slot a 0.90 e 23.3 a 0.98, cioè attraversa 1.3 e 5.8 linee. Dipende dal fatto che l'indirizzo successivo è calcolato  $((h+1) \& \text{mask})$  e non letto: senza catena di dipendenze la CPU sovrappone gli accessi, e il costo marginale è limitato dal throughput anziché dalla latenza. L'unordered\_map non può farlo, dovendo leggere il puntatore prima di proseguire.
- Il valore non dipende dal riempimento 0.5:  $C_{\text{seguito}}$  è misurato a 0.90-0.98. La contiguità è una proprietà del linear probing; il riempimento determina quanti accessi successivi si eseguono, non il costo di ciascuno.
- Limiti:  $C_{\text{seguito}}$  costante è un'assunzione, verificata solo nella regione satura, dove due pendenze locali indipendenti danno 1.210 e 1.124. Sotto 0.90 non esiste evidenza diretta, ma il termine

pesa l'1.9% del totale a 0.10 e il 5.6% a 0.50, quindi anche raddoppiandolo C\_primo cambierebbe del 2-6%. Controllo indipendente: 0.95 non entra nella stima e torna entro il 2%.

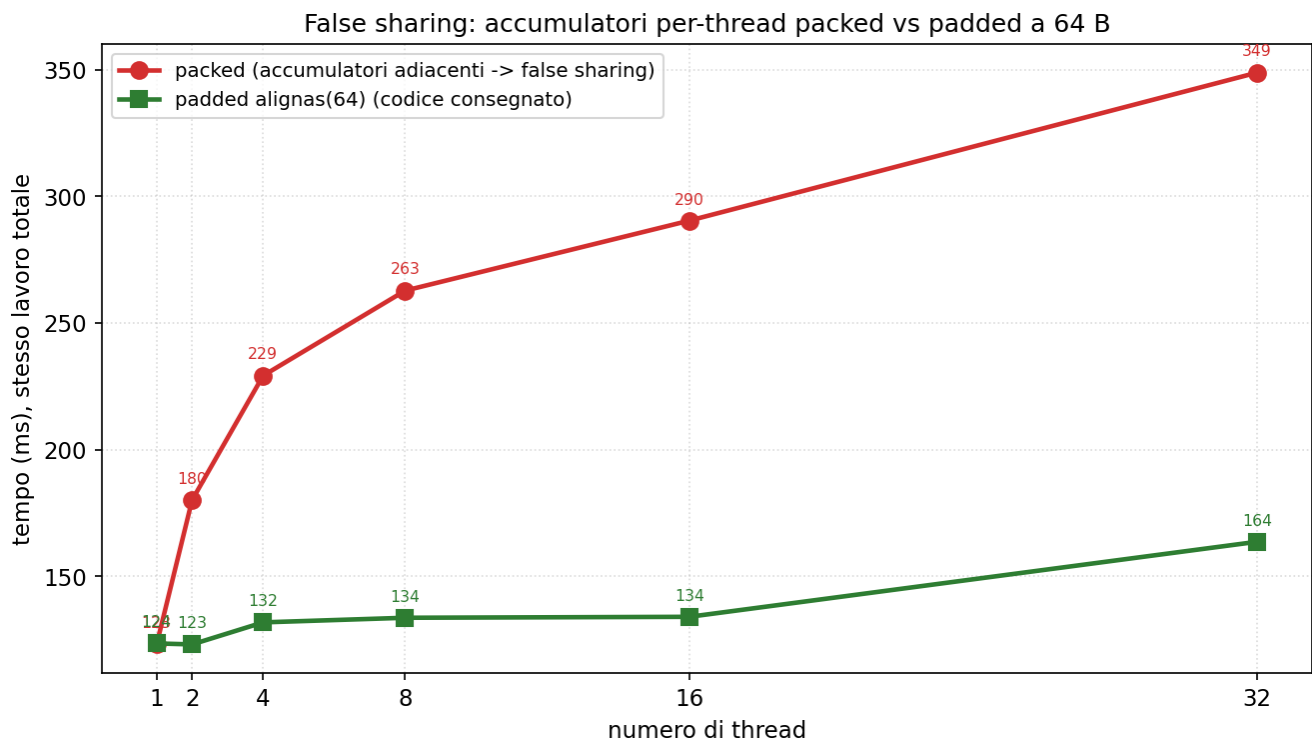
## Esp. 2: FlatCountMap



Costo del probe al crescere della tabella: oltre L3 diventa DRAM-bound.

- Il probe passa da 3.8 ns (64 KB, in L2) a 5.8 ns (16 MB, in L3) a 14.4 ns (1 GB, in DRAM).
- Oltre L3 il divario con l'unordered\_map si amplia (11 ns in cache, 63 ns in DRAM): ogni accesso è un miss a piena latenza, e la FlatCountMap ne paga uno solo per lookup. Non perché il prefetcher anticipi lo scorrimento, ma perché a questo riempimento (0.25, per via del sizing x2 e dei duplicati di R) i probe contati sono 1.167: almeno l'83% delle lookup tocca un solo slot, quindi non c'è quasi niente da scorrere. L'unordered\_map invece, per la stessa lookup, tocca il bucket e poi uno o più nodi sparsi sull'heap.
- Per questo P è scelto in modo che ogni tabella per-partizione resti in L3 (circa 1 MB a P=512).

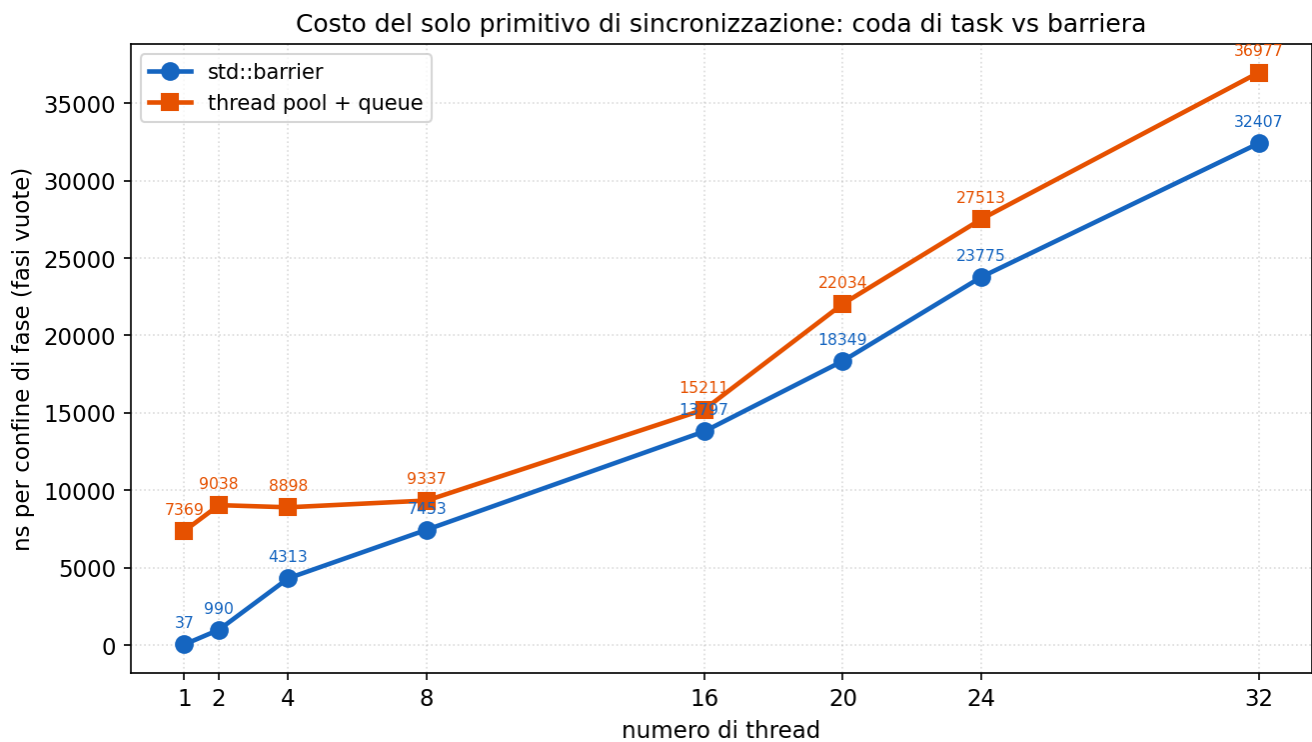
## Esp. 2: FlatCountMap



False sharing: accumulatori packed vs padded a 64 B.

- Con accumulatori adiacenti (packed) lo stesso lavoro passa da 123 ms (1 thread) a 349 ms (32 thread); con il padding a 64 B resta piatto (circa 130 ms), fino a 2.2x di penalità.
- Il false sharing si manifesta quando due accumulatori condividono una cache line e si invalidano a vicenda a ogni scrittura.
- Nel codice consegnato il padding è difensivo: ogni thread accumula in una variabile locale e scrive sull'array condiviso una sola volta a fine fase, per cui il false sharing effettivo è trascurabile.

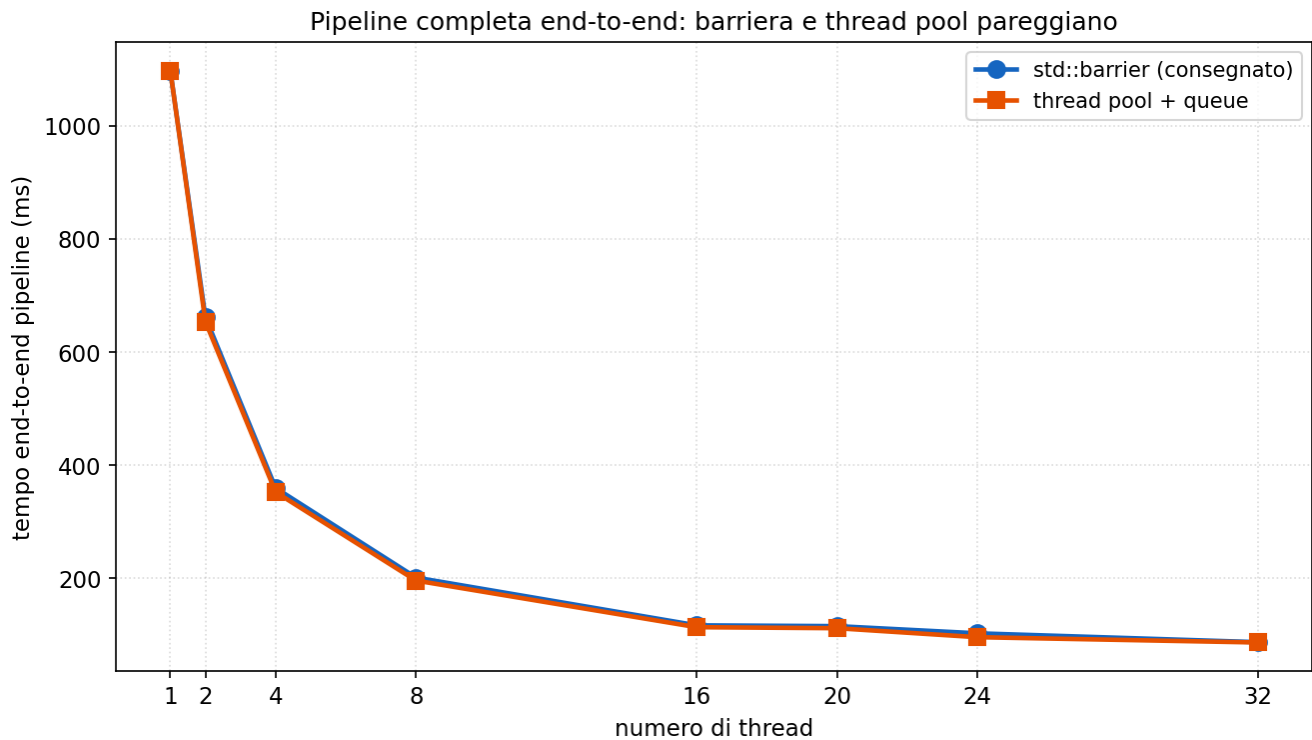
### Esp. 3: barriera vs thread pool



Costo del solo primitivo di sincronizzazione (fasi vuote).

- Con fasi vuote, per isolare la sola sincronizzazione: la barriera varia da 37 ns (1 thread) a 32 microsecondi (32 thread), la coda di task da 7.4 a 37 microsecondi.
- La coda è costantemente più costosa (198x a 1 thread, circa 1.1x a 32): paga submit e wakeup che la barriera non richiede.
- I valori sono nell'ordine dei microsecondi: il fattore 198x è relativo, mentre l'overhead assoluto resta minimo.

### Esp. 3: barriera vs thread pool

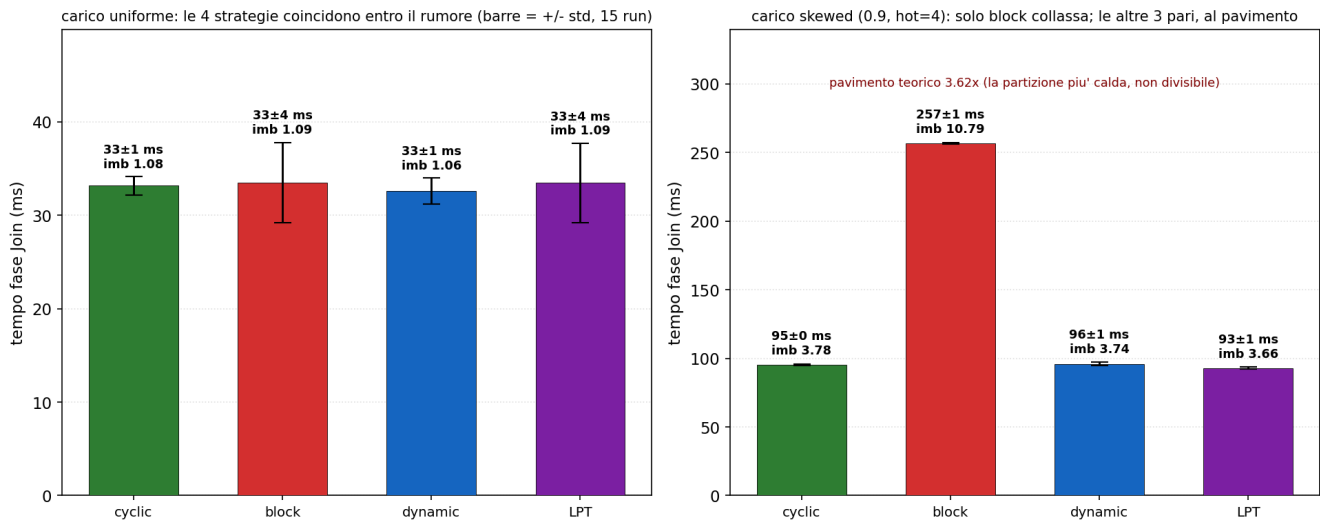


Pipeline completa end-to-end: barriera e thread pool pareggiano.

- Sulla pipeline completa i due primitivi risultano equivalenti end-to-end (87.0 vs 86.5 ms a  $p=32$ ), con identico `join_count`: varia solo il primitivo.
- La ragione è che il lavoro delle fasi è nell'ordine dei millisecondi e la sincronizzazione dei microsecondi: il fattore 198x non incide su un totale in ms.
- Il report non sostiene che la barriera sia più veloce, ma che una barriera è comunque necessaria a ogni confine (per le dipendenze fra fasi), rendendo il thread pool una complessità superflua. Vale lo stesso principio del cyclic: a parità di prestazioni si adotta la soluzione più semplice.

## Esp. 4: distribuzione del carico nel join

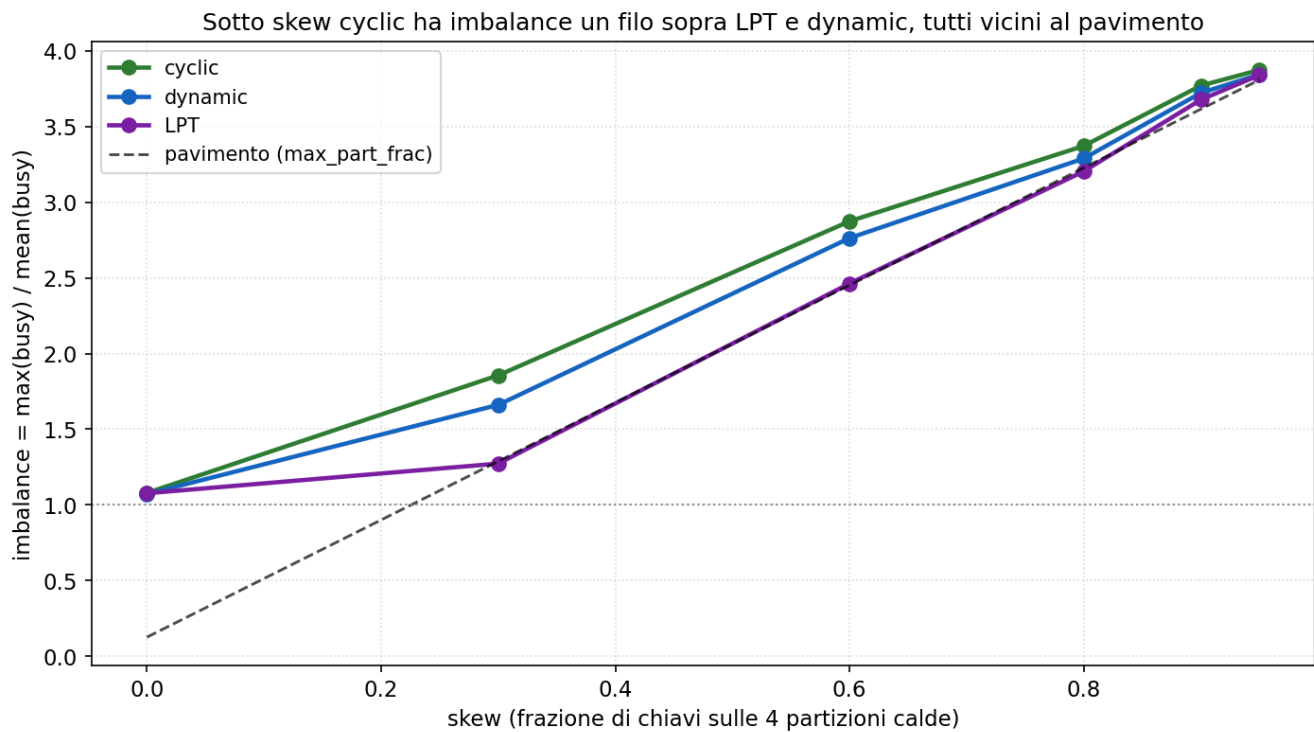
Distribuzione del carico nella fase Join (NR=10M, P=128, t=16, node Ivy Bridge)



Quattro strategie con barre d'errore: uniforme vs skew.

- Quattro strategie sulla sola fase join, con barre d'errore su 15 ripetizioni.
- Sul carico del Modulo 2 (uniforme) le quattro coincidono: imbalance circa 1.08, tempi entro il rumore di misura. La strategia è ininfluente.
- Sotto carico skewed (stress test, tipico del Modulo 3) solo block degrada (imbalance 10.8); cyclic, dynamic e lpt restano prossime al pavimento (circa 3.6).
- Il pavimento 3.62 è un limite inferiore imposto dai dati, non dall'algoritmo. Con skew 0.9 la partizione più calda contiene da sola circa il 22.6% del lavoro, contro una quota equa del 6.25% per thread (1/16):  $22.6/6.25 = 3.62$ . Poiché una partizione è indivisibile (elaborata da un solo thread), il thread che la riceve svolge almeno il 22.6% del lavoro, e nessuna strategia può scendere sotto tale valore.

#### Esp. 4: distribuzione del carico nel join



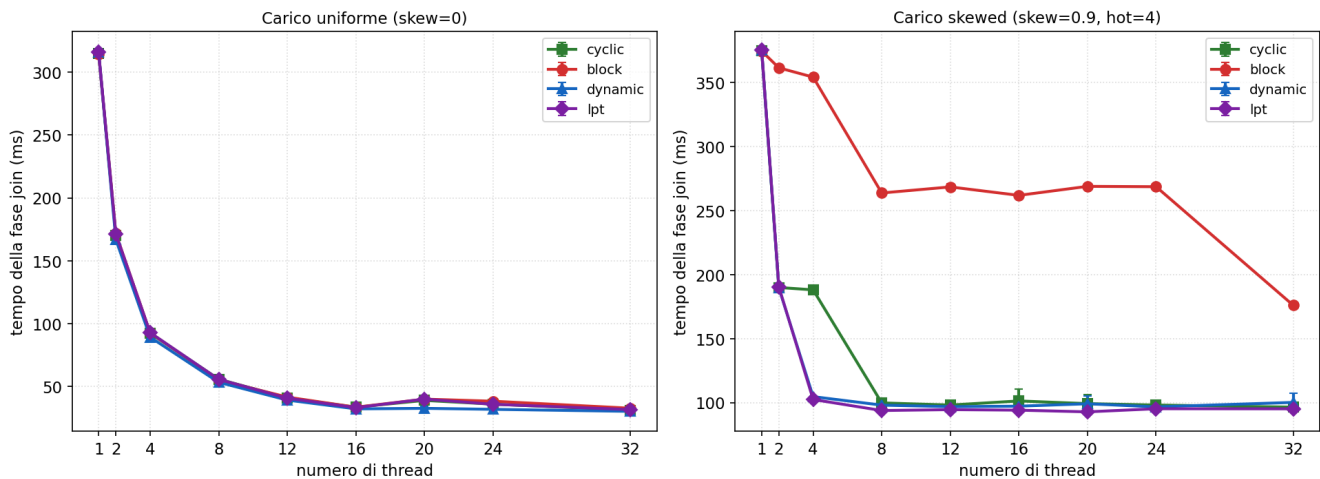
Imbalance al crescere dello skew.

- A skew 0.9: LPT 3.66, dynamic 3.74, cyclic 3.78, tutte prossime al pavimento 3.62.
- Sotto skew cyclic è marginalmente peggiore (circa 3%), sotto carico uniforme è pari alle altre: non è la più veloce, ma non è richiesto esserlo sul carico uniforme del Modulo 2.
- Il carico skewed appartiene comunque al Modulo 3: il generatore del Modulo 2 produce chiavi uniformi.



## Esp. 4: distribuzione del carico nel join

Tempo della fase join vs numero di thread, per strategia (NR=10M, P=128, node Ivy Bridge)



Tempo della fase join vs numero di thread, per strategia.

- Sul carico uniforme (Modulo 2) le partizioni hanno costo pressoché uguale, quindi qualsiasi assegnamento bilanciato è equivalente: le quattro strategie coincidono fino a 16 core e a 32, scalando insieme (315 ms a 1 thread, 31 ms a 32). Nella fascia Hyper-Threading (20-24 thread) solo dynamic migliora leggermente, per il ribilanciamento a runtime; cyclic resta pari alle altre strategie statiche (block, lpt).
- Il pannello destro riporta il carico skewed: block resta indietro (circa 260 ms fra 8 e 24 thread), mentre cyclic, dynamic e lpt scendono al pavimento di circa 95 ms. cyclic presenta uno scarto solo a 4 thread (188 vs 103), poi converge.
- cyclic è scelta perché al punto di lavoro (fino a 16 core) eguaglia le altre ed è la più semplice (una formula, senza atomiche né ordinamento); l'unica da escludere è block. Non vi è contraddizione con il report, che si riferisce al carico uniforme.

## Esp. 5: histogram memory-bound



Banda del nodo: read pura e histogram raggiungono lo stesso tetto a 16 core.

- L'esperimento dimostra che l'histogram è memory-bound, cioè limitato dalla banda di memoria e non dal calcolo, il che ne spiega la scarsa scalabilità.
- La lettura pura satura a 41 GB/s (il massimo erogato dalla memoria a 16 core); l'histogram parte da 4.7 GB/s a 1 core e raggiunge 40 GB/s a 16 core, cioè lo stesso tetto.
- Raggiungere la banda di una lettura pura indica che il limite è la memoria e che il calcolo (la hash) è nascosto: con perf l'histogram esegue circa 11 istruzioni per record contro 2.75 della lettura pura, ottenendo tuttavia la stessa banda.

## Esp. 5: histogram memory-bound

Roofline: l'histogram è memory-bound per ogni convenzione di conteggio (0.125 e ~0.6 stanno sulla diagonale)

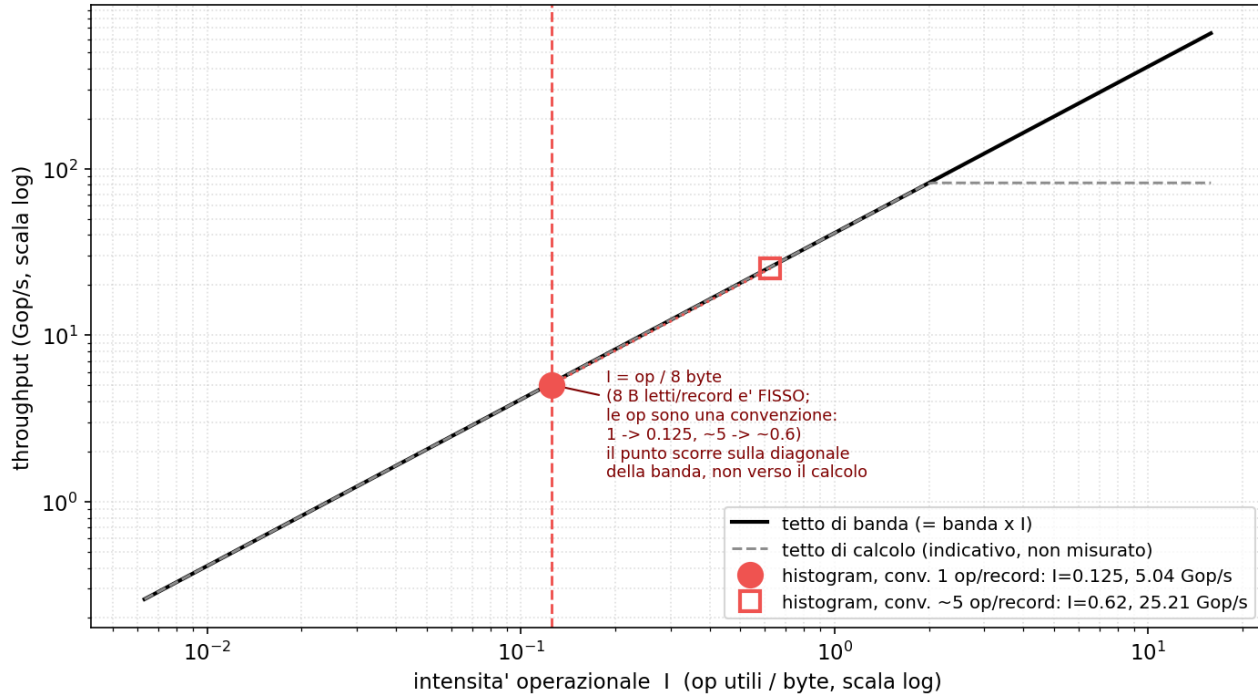
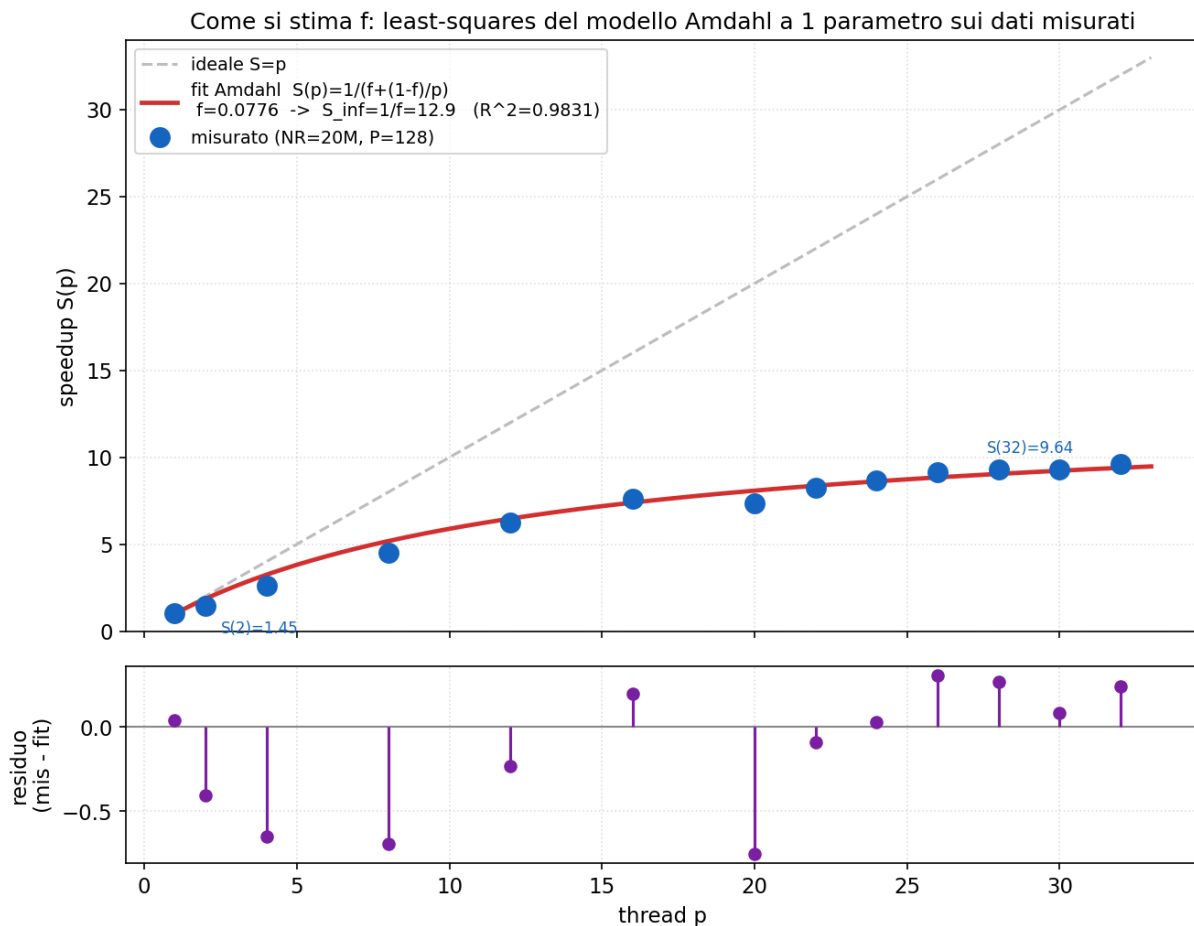


Diagramma roofline: l'histogram è memory-bound sia a  $I=0.125$  (1 op/record) sia a  $I \approx 0.62$  (contando le ~5 op della hash); i due punti stanno sulla stessa diagonale della banda, non verso il tetto di calcolo.

- Il roofline indica cosa limita un kernel, il calcolo della CPU o la banda di memoria, in funzione dell'intensità operativa  $I$  (operazioni utili per byte letto).
- L'histogram ha  $I = 0.125$ , cioè quasi nessun calcolo per byte: si colloca nella regione memory-bound, dove il limite è  $I \times \text{banda}$  e non il picco di calcolo.
- Ne segue che aggiungere core non porta beneficio una volta saturata la banda (condivisa): per questo l'histogram scala solo 2-3x fino a  $p=32$ .

## Esp. 6: legge di Amdahl



Fit del modello di Amdahl sulla curva di speedup misurata.

- Amdahl:  $S(p) = 1/(f + (1-f)/p)$ , con  $f$  la frazione seriale; per  $p$  tendente all'infinito  $S$  tende a  $1/f$ , il limite di speedup.
- La  $f$  si stima dai dati, non si conta dal codice. Il modello ha un solo incognito,  $f$ : si cerca il valore per cui la curva  $1/(f + (1-f)/p)$  passa il più vicino possibile ai punti di speedup misurati, cioè quello che minimizza la somma dei quadrati degli scarti tra misura e modello su tutti i thread count insieme (è ciò che fa `curve_fit`). Sui dati NR=20M, P=128 esce  $f = 0.078$ , quindi  $S_{\infty} = 1/f = 12.9$ .
- Che un valore preciso esca dai dati si vede girando il modello al reciproco:  $1/S(p) - 1/p = f \cdot (1 - 1/p)$ . È una retta per l'origine di pendenza  $f$ , quindi ogni thread count misurato dà già una stima  $f = (1/S - 1/p)/(1 - 1/p)$ ; ai  $p$  alti si assestano intorno a 0.078 (0.074 a  $p=16$ , 0.077 a  $p=24$ , 0.075 a  $p=32$ ). Il fit ai minimi quadrati non fa che combinare tutte queste stime in un unico valore, pesando di più i  $p$  alti dove lo speedup è grande.
- Il valore unico è il minimo della funzione errore totale  $E(f)$ , che dipende solo da  $f$  ( $p$  e  $S$  sono i dati fissi):  $E(f) = \text{somma su tutti i } p \text{ di } [S_{\text{misurato}}(p) - 1/(f + (1-f)/p)]^2$ . È una conca con un solo fondo, e quel fondo è  $f$ . Sui dati NR=20M, P=128 (calcolata da `plot_amdahl.py`):

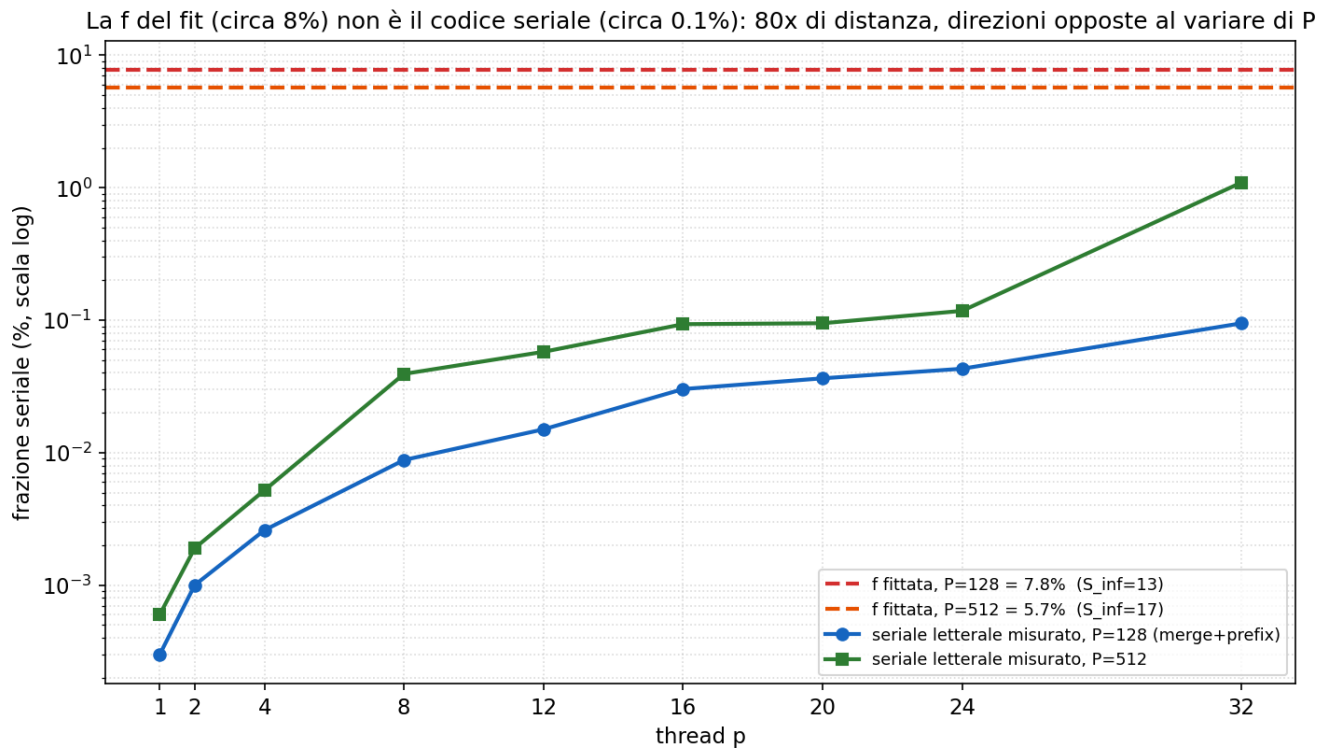
|                               |                               |                     |
|-------------------------------|-------------------------------|---------------------|
| $f=0.020 \rightarrow E=548.7$ | $f=0.078 \rightarrow E= 1.97$ | $\leftarrow$ minimo |
| $f=0.050 \rightarrow E= 58.2$ | $f=0.085 \rightarrow E= 4.11$ |                     |
| $f=0.070 \rightarrow E= 4.85$ | $f=0.094 \rightarrow E= 11.1$ |                     |
|                               | $f=0.120 \rightarrow E= 45.1$ |                     |

- $f$  è unico perché la conca ha un solo fondo, non perché i punti convergano ai  $p$  alti; `curve_fit` scende

lungo la pendenza da 0.05 fino a dove  $dE/df = 0$  e trova  $f = 0.078$ , quindi  $S_{\infty} = 1/f = 12.9$ .

- $R^2 = 0.983$  misura quanto la curva a un parametro spiega la variazione dei dati: il 98.3% dello scarto dei punti è catturato dalla forma di Amdahl, quindi  $f$  è un riassunto reale della curva e non un numero forzato.
- La  $f$  non è comunque ricavabile dal codice, perché la frazione seriale di Amdahl è un parametro effettivo che aggrega tutte le cause di scalabilità non ideale (banda, sincronizzazione, sbilanciamento) e non solo le righe seriali. È apparente: non corrisponde al codice seriale, come mostra il grafico successivo.

## Esp. 6: legge di Amdahl



La frazione seriale del fit contro il codice seriale misurato.

- Il codice effettivamente seriale (merge + prefix sum, cronometrato) è lo 0.095% del tempo a  $p=32$ , 80 volte inferiore alla f del fit (7.8%): la f del fit non corrisponde dunque al codice seriale.
- Tale f incorpora la saturazione di banda: le fasi memory-bound cessano di scalare, e il modello a un parametro attribuisce l'intero effetto a f.
- La conferma è nelle direzioni opposte: al crescere di P la f del fit diminuisce (0.078 a  $P=128$ , 0.058 a  $P=512$ ), mentre il codice seriale letterale aumenta (dallo 0.095% all'1.09%). La f misura quindi la banda, non righe di codice seriale.